
32-bit 5-stage Pipelined Datapath

RISC-V ISA

Full Name

Dư Đức Minh

June 6, 2026

Contents

1	Introduction	2
2	Overall Architecture	2
3	Instruction Execution	4
3.1	R-Type and I-Type Instructions	4
3.2	Load and Store Instructions	10
4	Multiplication and Division (Mul/Div) Extension	13
4.1	Overall flow of execution	13
4.2	Setup stage	13
4.3	Multiplication Algorithm	14
4.4	Division Algorithm	14
4.5	Control signals	20
5	Data Hazards	23
6	Simulation and Implementation	24
6.1	Simulation	24
6.2	Implementation	28
7	Conclusion	29

1 Introduction

This project explores the structural design and implementation of a processor architecture based on the RISC-V Instruction Set Architecture (ISA). RISC-V is an open-source ISA that has gained immense popularity in both academia and industry due to its modular, extensible, and highly efficient design. The core focus of this project is the implementation of a pipelined datapath capable of executing basic instructions alongside the M-extension (multiplication and division) instructions. Instruction execution is divided into five stages: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). This pipelining significantly improves processor performance by allowing multiple instructions to be processed concurrently.

In addition, this design includes a dedicated hazard detection unit that performs data forwarding to eliminate stall penalties when data dependencies occur. Beside, since the extended multiplication and division instructions require multiple clock cycles in the Execute stage, the design introduces specialized logic blocks to manage the pipeline control. These blocks detect necessary stalls and arrange the program flow efficiently. Notably, the architecture allows independent instructions to be processed concurrently alongside a division operation, ensuring the entire pipeline does not freeze while waiting for the multi-cycle divider to finish.

2 Overall Architecture

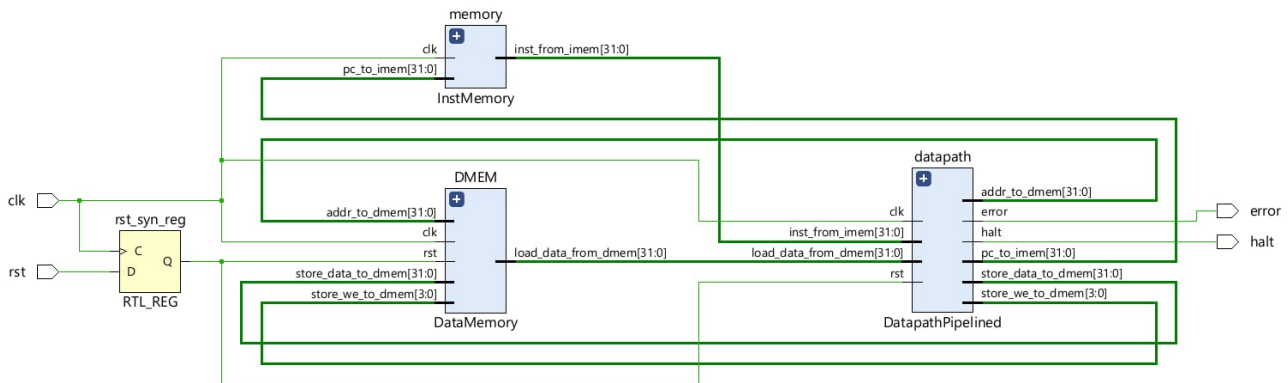


Figure 1: Overall Processor Architecture

The processor design is about the datapath connect with two memory (instruction memory and data memory).

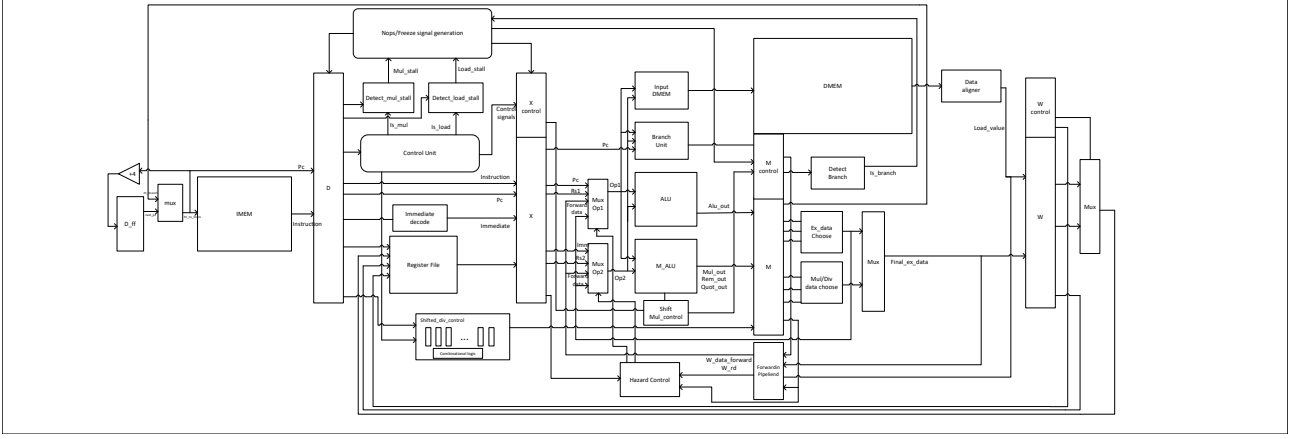


Figure 2: Overall Pipelined Datapath Architecture

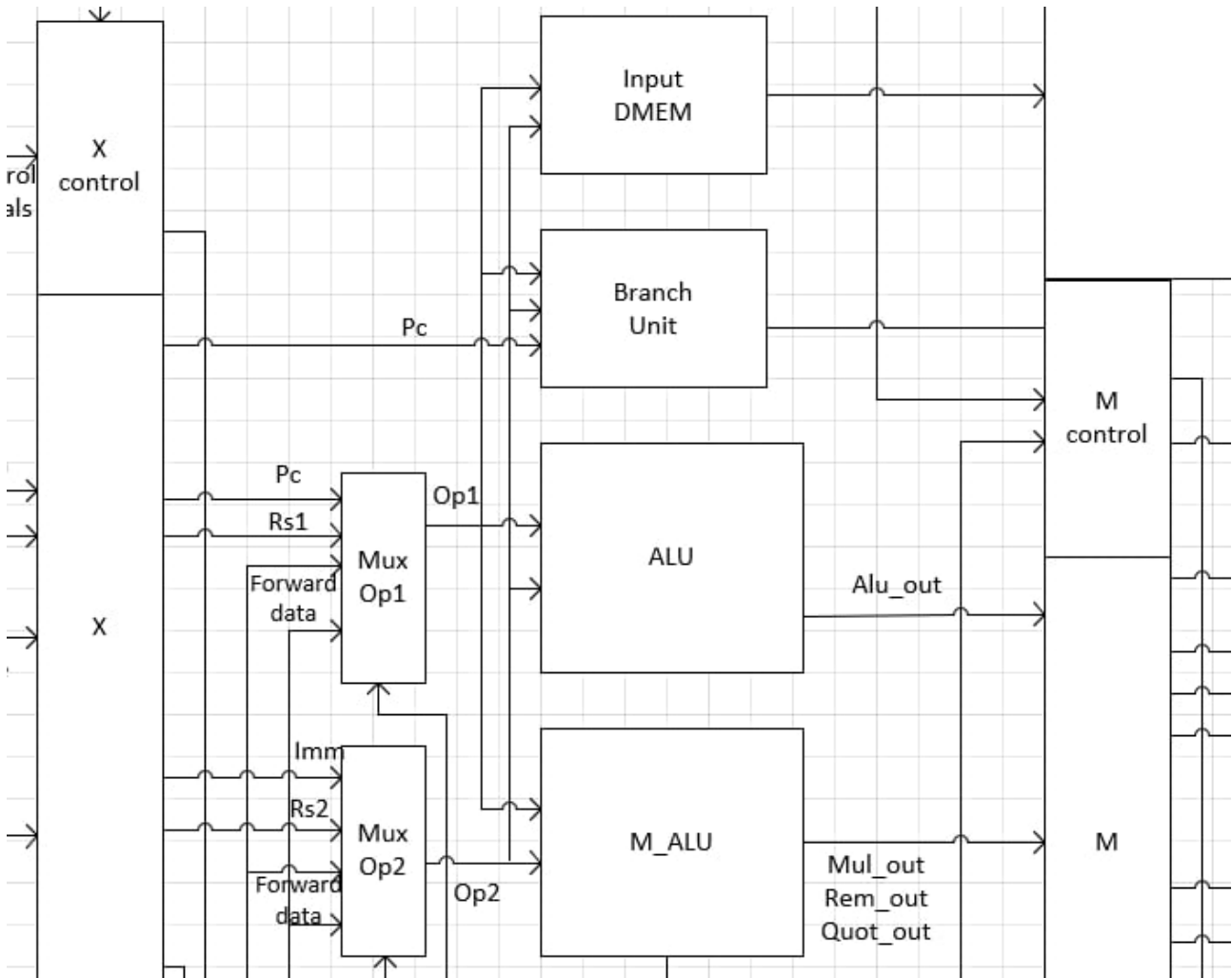
The provided diagram illustrates the internal modules within each stage of the datapath and their interconnections. The dataflow initiates in the Instruction Fetch (IF) stage, where an instruction is retrieved from Instruction Memory using an address selected between the incremented Program Counter ($PC+4$) and a calculated branch target. At the positive clock edge, both the fetched instruction and the current PC are latched into the IF/ID pipeline register to enter the Instruction Decode (ID) stage. Here, the instruction is parsed into its constituent fields (opcode, rd, funct3, rs1, rs2, and funct7). These fields are routed to their respective modules to read source operand data, generate sign-extended immediate values, and produce the necessary control signals. Crucially, to minimize stall penalties, the hazard detection block is also located in the ID stage to identify dependencies and insert stalls or NOPs as early as possible.

Following the decode phase, the pipeline transitions to the Execute (EX) stage. The primary function of this stage is to prepare and select the correct operands—often utilizing data forwarding logic—before routing them into the execution blocks. The EX stage houses several core components: the main ALU for standard arithmetic, an M_ALU dedicated to multiplication and division, and a Branch Unit responsible for evaluating branch conditions and target addresses. Additionally, dedicated control modules (`shifted_div_control` and `shift_mul_control`) manage the multi-cycle M-extension instructions, ensuring the correct sequence of control signals is fed into the EX/MEM pipeline register. Finally, the instruction progresses through the Memory Access (MEM) and Write Back (WB) stages. The MEM stage handles Data Memory operations and formats the loaded data, while the WB stage utilizes a series of multiplexers to select the final result—whether from the ALU, M_ALU, or memory—to be written back into the Register File.

3 Instruction Execution

The supported instruction set divides instructions into several distinct categories based on their functional behavior: R-type, I-type, Load/Store, and Branch/Jump instructions.

3.1 R-Type and I-Type Instructions



R-type and I-type instructions execute basic arithmetic and logical operations through the ALU module, after which the computed results are written back to the register file. For the organization of this report, the LUI and AUIPC instructions are also discussed within this section. The primary focus here is the structural implementation of the ALU module itself. Notably, the design incorporates a Carry Lookahead Adder (CLA) to efficiently execute addition operations within the ALU.

ALU module

The primary ALU module is structurally designed to compute eight arithmetic and logical operations concurrently. Rather than computing only one requested operation, the ALU evaluates all possibilities in parallel, and a multiplexer driven by the control signal selects the appropriate result to propagate forward. Auxiliary there are other signals surely the execute the right operation.

```
1 The primary ALU module is structurally designed to compute eight arithmetic
  and logical operations concurrently. Rather than computing only one
  requested operation, the ALU evaluates all possibilities in parallel, and
  a multiplexer driven by the alu_control signal selects the appropriate
  result to propagate forward. Auxiliary control signals are also routed to
  the datapath to ensure accurate operational behavior.
2 module ALUs (
3     input  logic ['REG_SIZE:0] rs1, rs2,
4     input  logic inst_type, // 0: R-type ; 1: I-type
5     input  logic auipc,
6     input  logic [3:0] control, // control[3] = inst[30], remain 3 bits
  are funct3
7     output logic ['REG_SIZE:0] alu_out
8 );
9     logic sub_control ;
10    logic sra_control ;
11    assign sub_control = ~inst_type && control[3] ;
12    assign sra_control = control[3] ;
13
14    logic ['REG_SIZE:0] data2;
15    logic ['REG_SIZE:0] adder_out;
16    logic cout;
17    assign data2 = sub_control? ~(rs2) : rs2 ;
18    cla unit(
19        .a(rs1),
20        .b(data2),
21        .cin(sub_control),
22        .sum(adder_out),
23        .carry_out(cout)
24    );
25
26    logic ['REG_SIZE:0] alu_add_sub ;
27    logic ['REG_SIZE:0] alu_sll ;
28    logic ['REG_SIZE:0] alu_slt ;
29    logic ['REG_SIZE:0] alu_sltu ;
30    logic ['REG_SIZE:0] alu_xor ;
31    logic ['REG_SIZE:0] alu_sr ;
32    logic ['REG_SIZE:0] alu_or ;
33    logic ['REG_SIZE:0] alu_and ;
34    always_comb
35    begin
36        alu_add_sub = adder_out ;
37        alu_sll = rs1 << {rs2[4:0]} ;
```

```

38     alu_slt = $signed(rs1) < $signed(rs2) ? 32'd1 : 32'b0 ;
39     alu_sltu = rs1 < rs2 ? 32'd1 : 32'b0 ;
40     alu_xor = rs1 ^ rs2 ;
41     alu_sr = sra_control? ((rs1 >> rs2[4:0]) | ({32{rs1[31]}} << (32-rs2
[4:0]))) :
42                                     (rs1 >> rs2[4:0]) ;
43     alu_or = rs1 | rs2 ;
44     alu_and = rs1 & rs2 ;
45 end
46
47 always_comb
48 begin
49     unique case (control[2:0] & (~{3{auipc}}))
50         3'b000: begin // ADD, SUB
51             alu_out = alu_add_sub ;
52         end
53         3'b001: begin // SLL
54             alu_out = alu_sll ;
55         end
56         3'b010: begin // SLT
57             alu_out = alu_slt ;
58         end
59         3'b011: begin // SLTU
60             alu_out = alu_sltu ;
61         end
62         3'b100: begin // XOR
63             alu_out = alu_xor ;
64         end
65         3'b101: begin // SRL, SRA
66             alu_out = alu_sr ;
67         end
68         3'b110: begin // OR
69             alu_out = alu_or ;
70         end
71         3'b111: begin //AND
72             alu_out = alu_and ;
73         end
74         default: alu_out = 32'b0 ;
75     endcase
76 end
77 endmodule

```

Operand choose

Both R-type and I-type instructions utilize this ALU for computation. The primary distinction between the two formats lies in the selection of the second operand (Operand 2). A dedicated control signal determines whether Operand 2 receives the base register value or a sign-extended immediate value. Similarly, the first operand (Operand 1) utilizes a control signal to select between the base register value and the current Program Counter (PC) value, which is specifi-

cally required to execute the AUIPC instruction. Notably, the "base value" for both operands is determined by a preliminary selection between the original source register data and forwarded data from subsequent pipeline stages.

```

1 module choice_operand (
2     input  logic [1:0] op_control,
3     input  logic ['REG_SIZE:0] rs,
4     input  logic ['REG_SIZE:0] mem_forward,
5     input  logic ['REG_SIZE:0] wb_forward,
6     output logic ['REG_SIZE:0] operand
7 );
8     // control: 0x: rs ; 10: mem ; 01: wb
9     logic ['REG_SIZE:0] forward_op ;
10    assign forward_op = (op_control[0])? wb_forward : mem_forward ;
11    assign operand = (!op_control)? forward_op : rs ;
12 endmodule

```

```

1 choice_operand op1(
2     .op_control(op1_control),
3     .rs(rs1_fromX),
4     .mem_forward(ex_data),
5     .wb_forward(forward_data),
6     .operand(base_op1)
7 );
8 assign operand1 = (op1_choose_fromX)? pc_fromX : base_op1 ;
9
10 choice_operand op2(
11     .op_control(op2_control),
12     .rs(rs2_fromX),
13     .mem_forward(ex_data),
14     .wb_forward(forward_data),
15     .operand(base_op2)
16 );
17 assign operand2 = (op2_choose_fromX)? imm_fromX : base_op2 ;

```

Choose execute value

The concluding operation within this segment of the datapath requires a multiplexer to select between the final ALU result and the immediate data associated with the LUI instruction. In addition, the selection needs to select from one more value that is return address which corresponding to jal and jalr instruction. To mitigate timing bottlenecks and optimize the critical path of the Execute (EX) stage, the hardware implementation deliberately shifts this multiplexing logic into the subsequent Memory Access (MEM) stage, thereby balancing the propagation delay across the pipeline.

```

1 always_comb
2     begin
3         case ({rd_choose_fromM[1], is_lui_fromM})
4             2'b00: ex_data = aluout_fromM ;

```

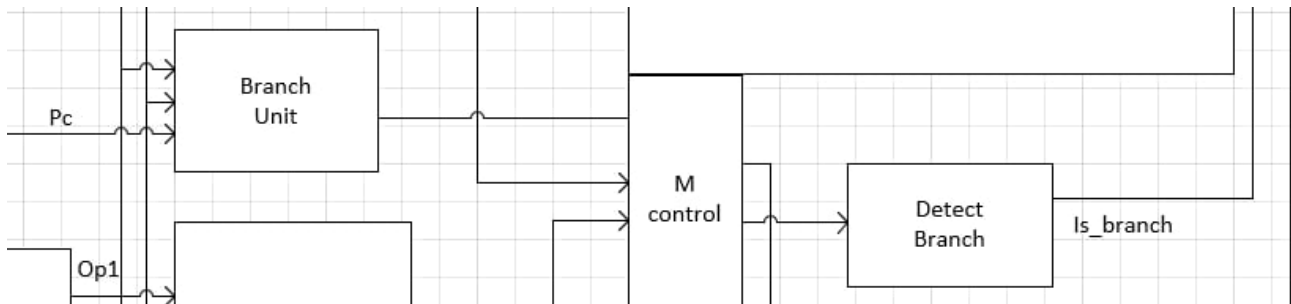


```

5      2'b01:  ex_data = lui_data_fromM ;
6      2'b10, 2'b11:  ex_data = ra_fromM ;
7      default:  ex_data = ra_fromM ;
8  endcase
9  end

```

Branch and Jump Instructions



To handle control flow instructions, the datapath is architected to calculate the branch target address and evaluate the branch condition logic entirely within the Execute (EX) stage. These resulting signals are latched through the EX/MEM pipeline register, and the final branch resolution (taken versus not taken) is executed in the Memory (MEM) stage. If the branch is determined to be "taken," a new instruction is fetched from the computed target address. Simultaneously, to recover from this control hazard, the design flushes the incorrect instructions that have already entered the pipeline by inserting NOPs (bubbles) into the IF/ID and ID/EX pipeline registers.

Compute target address

As outlined previously, the initial phase of control flow execution involves computing the branch target address. This is achieved using a multiplexer that selects the appropriate base address—either the current Program Counter (for standard branch and JAL instructions) or the first operand register value (for the JALR instruction). Subsequently, a Carry Lookahead Adder (CLA) computes the sum of this selected base value and the corresponding sign-extended immediate. As a final architectural safeguard, the least significant bit (LSB) of the resulting address is masked to 0 to strictly enforce word alignment and prevent address misalignment exceptions.

```

1  // base_addr is pc for branch and jal / rs1 for jalr
2  assign base_addr = (jump_fromX[1])? operand1 : pc_fromX ;
3  cla Br_Address (
4      .a(base_addr),
5      .b(imm_fromX),
6      .sum(tmp_pc),
7      .cin(1'b0),
8      .carry_out(e[0])

```

```

9      );
10     //set the last bit to 0 for JALR, remain branch already have the last
    bit is 0
11     assign pc_branch = {tmp_pc[31:1], 1'b0} ;

```

Branch unit

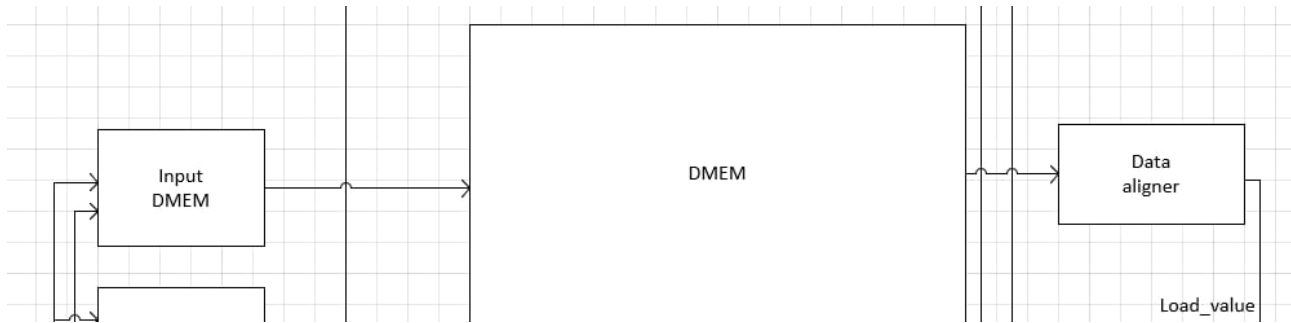
Concurrently, the Branch Unit module generates the **b_cond** (branch condition) signal. Similar to the ALU, this module performs six distinct comparisons in parallel and selects the correct result based on the specific instruction type. Afterward, combinational logic evaluates the **b_cond** signal alongside other control signals to assert the **is_branch** signal, which ultimately determines whether the branch is taken.

```

1 module Branch_condition(
2     input logic [2:0] branch_control,
3     input logic ['REG_SIZE:0] rs1,
4     input logic ['REG_SIZE:0] rs2,
5     output logic b_cond
6 );
7     always_comb begin
8         unique case(branch_control)
9             3'b000: b_cond = (rs1 == rs2) ;
10            3'b001: b_cond = (rs1 != rs2) ;
11            3'b100: b_cond = ($signed(rs1) < $signed(rs2)) ;
12            3'b101: b_cond = ($signed(rs1) >= $signed(rs2)) ;
13            3'b110: b_cond = (rs1 < rs2) ;
14            3'b111: b_cond = (rs1 >= rs2) ;
15            default: b_cond = 1'b0 ;
16        endcase
17    end
18 endmodule
19
20 module Detect_branch(
21     input logic b_cond,
22     input logic inst_jump,
23     input logic inst_branch,
24     output logic is_branch
25 );
26     (* max_fanout = 16 *) logic is_branch_reg ;
27     always_comb
28     begin
29         if(inst_jump) is_branch_reg = 1'b1 ;
30         else
31             begin
32                 is_branch_reg = (b_cond && inst_branch)? 1'b1 : 1'b0 ;
33             end
34         end
35     assign is_branch = is_branch_reg ;
36 endmodule

```

3.2 Load and Store Instructions



Input DMEM control

The Data Memory (DMEM) input address is computed within this module via a direct addition between the base address (Operand 1) and the immediate value (Operand 2). The module also incorporates a combinational logic block responsible for asserting the appropriate write-enable signals and formatting the data input for memory storage. To ensure architectural stability, a dedicated signal detects address misalignment based on the specific store instruction type. In the event of a misalignment, the write-enable signal is strictly deasserted to zero, blocking any memory write operations, as the current microarchitecture does not support unaligned memory accesses. Crucially, all output signals from this module are routed directly into the DMEM. By treating the DMEM block as the pipeline boundary, the design successfully circumvents the clock-to-Q propagation delays that would otherwise be introduced by intermediate flip-flops.

```

1 module Input_Mem_control(
2     input logic ['REG_SIZE:0] rs1,
3     input logic ['REG_SIZE:0] imm,
4     input logic [2:0] store_control,
5     input logic ['REG_SIZE:0] input_data,
6     output logic ['REG_SIZE:0] addr_to_dmem,
7     output logic ['REG_SIZE:0] store_data,
8     output logic [3:0] store_we,
9     output logic [1:0] load_byte,
10    output logic store_error
11);
12    assign addr_to_dmem = rs1 + imm ;
13    assign load_byte = addr_to_dmem[1:0] ;
14
15
16    logic [3:0] we_sb ;
17    logic [3:0] we_sh ;
18    logic [3:0] we_sw ;
19    logic s_misalign ;
20    assign s_misalign = (store_control[1] && addr_to_dmem[0])
21                      || (store_control[2] && |addr_to_dmem[1:0]) ;
22    always_comb begin
23        we_sb = (store_control[0])? (4'b0001 << addr_to_dmem[1:0]) : 4'b0 ;

```

```

24     we_sh = (store_control[1])? (4'b0011 << {addr_to_dmem[1], 1'b0}) :
4'b0 ;
25     we_sw = (store_control[2])? 4'b1111 : 4'b0 ;
26     store_we = {4{~(s_misalign)}} & (we_sb ^ we_sh ^ we_sw) ;
27     store_error = s_misalign ;
28 end
29
30 always_comb begin
31     unique case (store_control)
32         3'b001: store_data = {4{input_data[7:0]}} ;
33         3'b010: store_data = {2{input_data[15:0]}} ;
34         3'b100: store_data = input_data ;
35         default: store_data = 32'b0 ;
36     endcase
37 end
38 endmodule

```

Load value modify

For read operations, the DMEM supplies a complete 32-bit word to the datapath. To support the three variants of load instructions, the module processes the raw memory output—extracting and formatting the requested bytes—to formulate the correct value for Write Back to the Register File. Mirroring the store logic, load instructions are also evaluated for address misalignment. If an unaligned load is detected, the hardware asserts an error flag and suppresses the invalid memory read.

```

1 module Load_value(
2     input  logic [1:0] load_bytes,
3     input  logic [2:0] load_control,
4     input  logic ['REG_SIZE:0] load_from_dmem,
5     output logic ['REG_SIZE:0] load_value,
6     output logic load_error
7 );
8     logic [7:0] b_load ;
9     logic [15:0] h_load ;
10
11     always_comb
12     begin
13         load_error = 1'b0 ;
14
15         unique case (load_bytes)
16             2'b00: b_load = load_from_dmem[7:0] ;
17             2'b01: b_load = load_from_dmem[15:8] ;
18             2'b10: b_load = load_from_dmem[23:16] ;
19             2'b11: b_load = load_from_dmem[31:24] ;
20             default: b_load = 8'b0 ;
21         endcase
22
23         h_load = (!load_bytes)? load_from_dmem[31:16] : load_from_dmem[15:0]

```

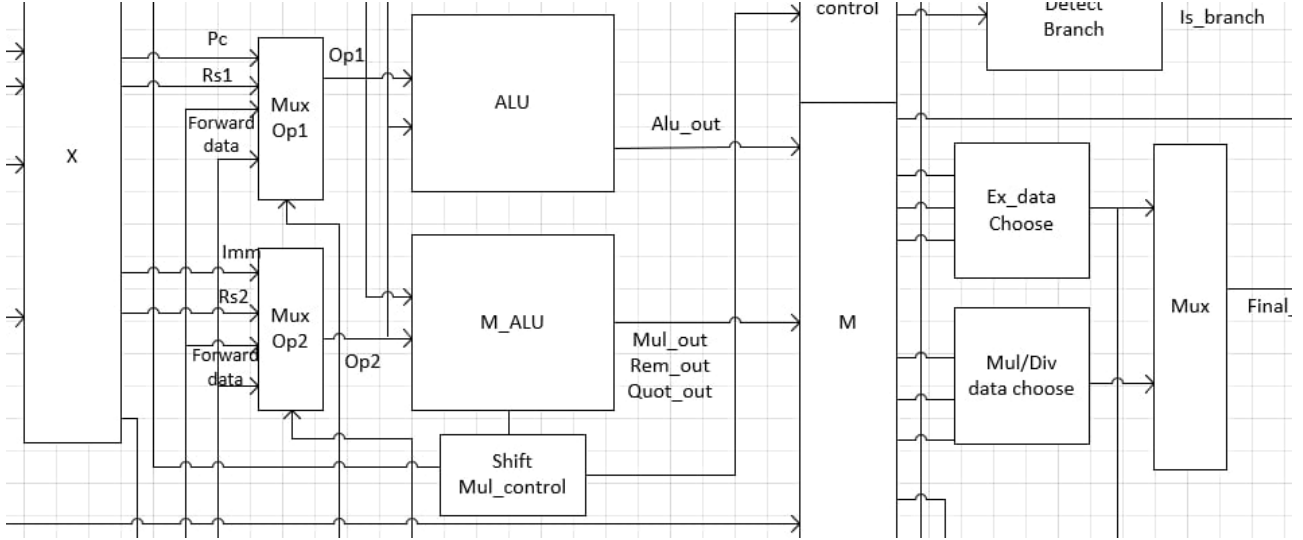
```

24 ;
25 unique case (load_control[1:0])
26 2'b00: load_value = (load_control[2])? {24'b0, b_load}
27           : {{24{b_load[7]}}}, b_load} ;
28 2'b01: begin
29     if(load_bytes[0])
30     begin
31         load_error = 1'b1 ;
32         load_value = 32'b0 ;
33     end
34     else load_value = (load_control[2])? {16'b0, h_load}
35           : {{16{h_load[15]}}}, h_load};
36 end
37 2'b10: begin
38     if(!load_bytes)
39     begin
40         load_error = 1'b1 ;
41         load_value = 32'b0 ;
42     end
43     else load_value = load_from_dmem ;
44 end
45 default: load_value = 32'b0 ;
46 endcase
47 end
48 endmodule
49

```

4 Multiplication and Division (Mul/Div) Extension

4.1 Overall flow of execution



Fundamentally, the datapath routing for M-extension instructions parallels that of standard R-type instructions. To handle these operations, an `M_ALU` module is instantiated concurrently alongside the primary ALU. Unlike standard arithmetic logic operations that resolve in a single clock cycle, multiplication and division involve inherent computational complexity, necessitating specialized multi-cycle hardware. The operational sequence consists of an initial setup stage followed by the algorithmic execution stage. Upon completion, the calculated result propagates through the Memory (MEM) and Write Back (WB) stages to update the Register File. Within the MEM stage, a dedicated multiplexing network selects the correct `m_alu_out` from the various M-extension functional units. The ultimate execution result is then resolved by a final multiplexer that selects between the M-extension output (`m_alu_out`) and the standard datapath result (`ex_data`).

4.2 Setup stage

Prior to the core algorithmic execution, both multiplication and division instructions require an initialization phase that consumes one clock cycle. For division operations, this setup stage evaluates the operation type and operand signs; if a signed division involves negative operands, they are actively converted to their absolute (positive) values. This architectural conversion is a strict prerequisite because the underlying division module employs an unsigned algorithm. For multiplication operations, the setup stage similarly conditions the data by sign-extending the input operands by one bit and configuring them into the appropriate signed or unsigned format required by the multiplier block.

```

1 logic ['REG_SIZE:0] pos_rs1, pos_rs2 ;
2   transform_pos #(.SIZE(32)) pos_op1 (

```

```

3      .in(rs1),
4      .out(pos_rs1)
5  );
6  transform_pos #(.SIZE(32)) pos_op2 (
7      .in(rs2),
8      .out(pos_rs2)
9  );
10
11  logic ['REG_SIZE:0] choice_dividend ;
12  logic ['REG_SIZE:0] choice_divisor ;
13  assign choice_dividend = ((control[0] == 0) & (rs1[31] == 1))? pos_rs1 :
rs1 ;
14  assign choice_divisor = ((control[0] == 0) & (rs2[31] == 1))? pos_rs2 :
rs2 ;
15
16  logic signed ['REG_SIZE+1:0] choice_mul1 ;
17  logic signed ['REG_SIZE+1:0] choice_mul2 ;
18  logic signed [('REG_SIZE+1)*2+1:0] signed_mul ;
19  assign choice_mul1 = (control[1:0] == 2'b11)? $signed({1'b0, rs1}) :
$signed(rs1) ;
20  assign choice_mul2 = ((control[1] == 1))? $signed({1'b0, rs2}) : $signed(
rs2) ;
21
22  always_ff @(posedge clk)
23  begin
24      dividend_in_divunit <= choice_dividend ;
25      divisor_in_divunit <= choice_divisor ;
26      mul_op1_in_mulunit <= choice_mul1 ;
27      mul_op2_in_mulunit <= choice_mul2 ;
28      delay_choose <= tmp_choose ;
29  end

```

4.3 Multiplication Algorithm

This design primarily utilizes the dedicated DSP slices inferred by the Vivado toolchain to execute multiplication operations. Because these built-in hardware blocks offer significantly lower logic complexity and much higher execution speeds compared to standard fabric implementations, utilizing DSP resources for this task is the optimal design choice.

```

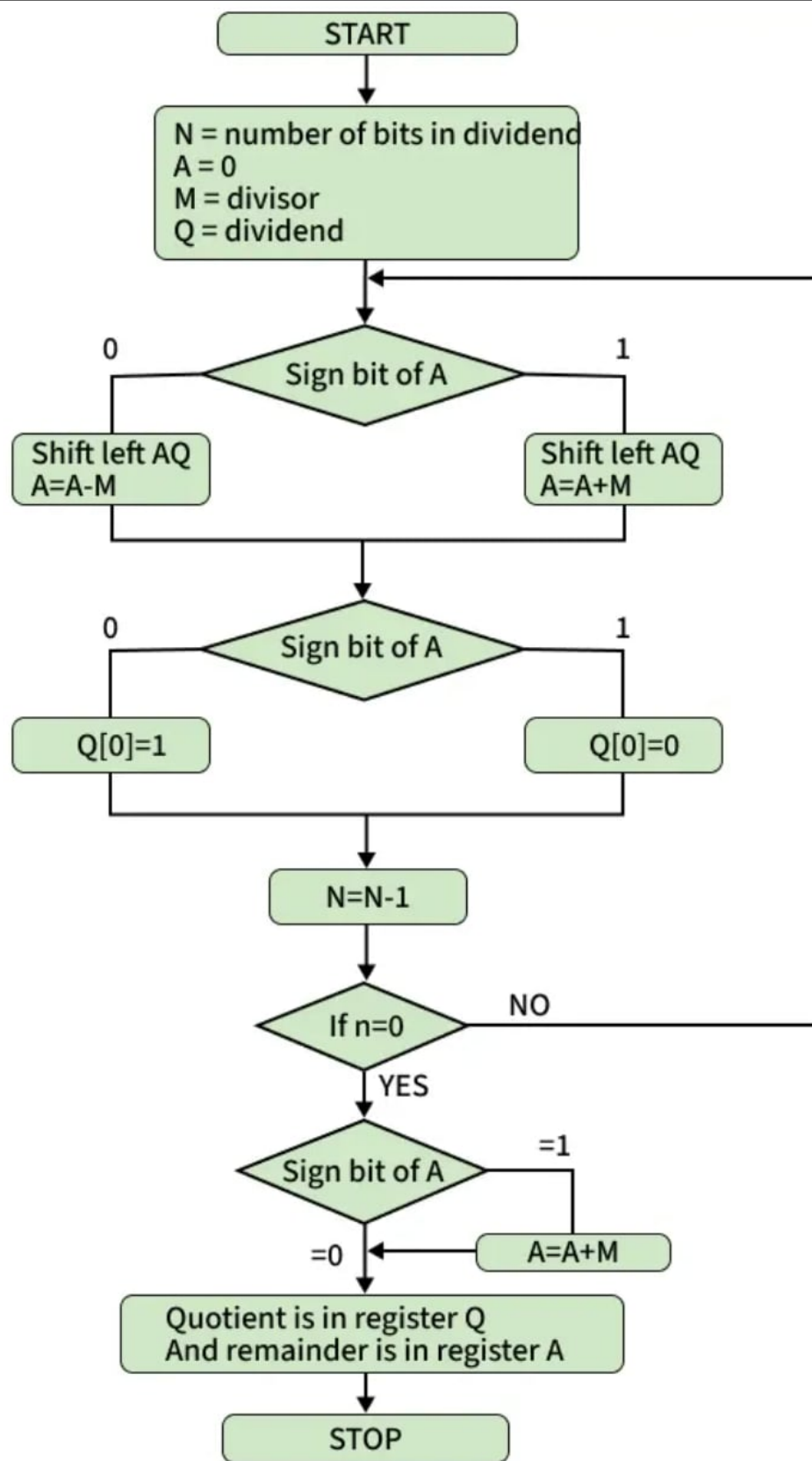
1 assign signed_mul = mul_op1_in_mulunit * mul_op2_in_mulunit ;
2 assign mul_out = signed_mul [('REG_SIZE+1)*2+1:0] ;

```

4.4 Division Algorithm

To handle division operations, the architecture implements a non-restoring unsigned division algorithm within the Execute (EX) stage. The execution sequence mandates an initial, single-cycle setup phase, which is distinct from the setup process discussed previously.. Following this initialization, the divider module computes one bit of the quotient per clock cycle, culminating

in the resolution of the final remainder during the last cycle. Consequently, this iterative division process requires a total of 33 clock cycles to complete execution.



```

2   input  logic clk, rst,
3   input  logic [2:0] i_signedchoose,
4   input  logic ['REG_SIZE:0] i_dividend,
5   input  logic ['REG_SIZE+1:0] i_divisor,
6   input  logic ['REG_SIZE+1:0] i_remainder,
7   output logic [2:0] o_signedchoose,
8   output logic ['REG_SIZE:0] o_dividend,
9   output logic ['REG_SIZE+1:0] o_divisor,
10  output logic ['REG_SIZE+1:0] o_remainder
11 );
12 always_ff @(posedge clk)
13 begin
14     if(rst)
15     begin
16         o_dividend  <= 'b0 ;
17         o_divisor   <= 'b0 ;
18         o_remainder <= 'b0 ;
19         o_signedchoose <= 'b0 ;
20     end
21     else begin
22         o_dividend  <= i_dividend ;
23         o_divisor   <= i_divisor ;
24         o_remainder <= i_remainder;
25         o_signedchoose <= i_signedchoose ;
26     end
27 end
28 endmodule
29
30 module divu_1iter (
31     input  logic ['REG_SIZE:0] i_dividend,
32     input  logic ['REG_SIZE+1:0] i_divisor,
33     input  logic ['REG_SIZE+1:0] i_remainder,
34     output logic ['REG_SIZE:0] o_dividend,
35     output logic ['REG_SIZE+1:0] o_remainder,
36     output logic ['REG_SIZE+1:0] o_divisor
37 );
38     logic bit_check ;
39     logic ['REG_SIZE+1:0] remainder_shifted ;
40     logic ['REG_SIZE:0] tmp_dividend ;
41
42     assign o_divisor = i_divisor ;
43     assign bit_check = i_remainder['REG_SIZE+1] ;
44     always_comb begin
45         case(bit_check)
46             1'b0: tmp_dividend = i_dividend | 'b1 ;
47             1'b1: tmp_dividend = i_dividend & ~( 'b1) ;
48             default: tmp_dividend = 'b0 ;
49         endcase
50
51         {remainder_shifted, o_dividend} = {i_remainder, tmp_dividend} << 1 ;
52

```

```

53         case(bit_check)
54             1'b0: o_remainder = remainder_shifted - i_divisor ;
55             1'b1: o_remainder = remainder_shifted + i_divisor ;
56             default: o_remainder = 'b0 ;
57         endcase
58     end
59 endmodule
60
61 module DividerUnsignedPipelined (
62     input  logic      clk, rst,
63     input  logic [2:0] i_signedchoose,
64     input  logic ['REG_SIZE:0] i_dividend,
65     input  logic ['REG_SIZE:0] i_divisor,
66     output logic ['REG_SIZE:0] o_remainder,
67     output logic ['REG_SIZE:0] o_quotient,
68     output logic [2:0] o_signedchoose
69 );
70     logic ['REG_SIZE:0] dividend_out_reg ['DIVIDER_STAGES-1] ;
71     logic ['REG_SIZE+1:0] divisor_out_reg ['DIVIDER_STAGES-1] ;
72     logic ['REG_SIZE+1:0] remainder_out_reg ['DIVIDER_STAGES-1] ;
73     logic ['REG_SIZE:0] dividend_in_reg ['DIVIDER_STAGES-1] ;
74     logic ['REG_SIZE+1:0] divisor_in_reg ['DIVIDER_STAGES-1] ;
75     logic ['REG_SIZE+1:0] remainder_in_reg ['DIVIDER_STAGES-1] ;
76     logic [2:0] signed_choose ['DIVIDER_STAGES] ;
77     assign signed_choose[0] = i_signedchoose ;
78
79     always_comb
80     begin
81         // Widen 1 bit divisor and remainder to deal with overflow when
82         calculate remainder
83         dividend_in_reg[0] = i_dividend << 1 ;
84         remainder_in_reg[0] = {32'b0, i_dividend['REG_SIZE]} - {1'b0,
85         i_divisor} ;
86         divisor_in_reg[0] = {1'b0, i_divisor} ;
87     end
88
89     genvar div_unit ;
90     genvar regs_unit ;
91     generate;
92         for(div_unit=0 ; div_unit < 'DIVIDER_STAGES-2 ; div_unit++)
93         begin : g_div_unit
94             divu_liter div_block(
95                 .i_dividend(dividend_out_reg[div_unit]),
96                 .i_divisor(divisor_out_reg[div_unit]),
97                 .i_remainder(remainder_out_reg[div_unit]),
98                 .o_dividend(dividend_in_reg[div_unit+1]),
99                 .o_remainder(remainder_in_reg[div_unit+1]),
100                 .o_divisor(divisor_in_reg[div_unit+1])
101             );
102         end
103     end

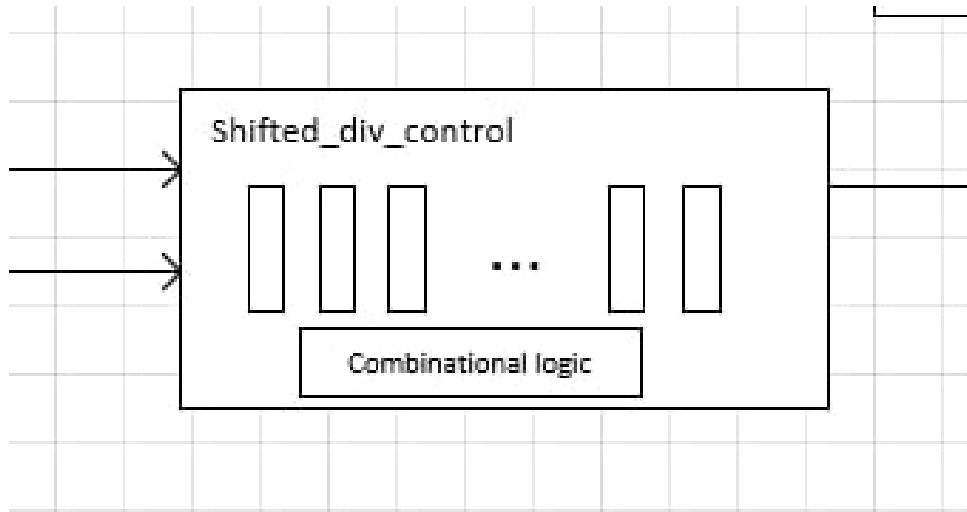
```

```

102     for(regs_unit=0 ; regs_unit < 'DIVIDER_STAGES-1 ; regs_unit++)
103     begin : g_div_pipeline
104         Regs pipelined(
105             .clk(clk),
106             .rst(rst),
107             .i_signedchoose(signed_choose[regs_unit]),
108             .i_dividend(dividend_in_reg[regs_unit]),
109             .i_divisor(divisor_in_reg[regs_unit]),
110             .i_remainder(remainder_in_reg[regs_unit]),
111             .o_signedchoose(signed_choose[regs_unit+1]),
112             .o_dividend(dividend_out_reg[regs_unit]),
113             .o_divisor(divisor_out_reg[regs_unit]),
114             .o_remainder(remainder_out_reg[regs_unit])
115         );
116     end
117 endgenerate
118
119 always_comb begin
120     if(remainder_out_reg['DIVIDER_STAGES-2']['REG_SIZE+1] == 0)
121     begin
122         o_quotient = dividend_out_reg['DIVIDER_STAGES-2] | 'b1;
123         o_remainder = remainder_out_reg['DIVIDER_STAGES-2']['REG_SIZE:0]
124     ;
125     end
126     else begin
127         o_quotient = dividend_out_reg['DIVIDER_STAGES-2] & ~('b1);
128         o_remainder= remainder_out_reg['DIVIDER_STAGES-2']['REG_SIZE:0] +
129             divisor_out_reg[
130 'DIVIDER_STAGES-2] ['REG_SIZE:0];
131     end
132     o_signedchoose = signed_choose['DIVIDER_STAGES-1] ;
133 end
134 endmodule

```

4.5 Control signals



For multiplication operations, the datapath incurs a minimal one-cycle stall for the setup phase, after which execution completes in the subsequent cycle, allowing standard program flow to resume. In contrast, division instructions dictate a substantial 34-cycle latency, comprising one setup cycle and 33 execution cycles. To manage the control flow across this multi-cycle latency, the architecture implements a 33-stage shift register chain. During the Instruction Decode (ID) stage, the control signals corresponding to the division instruction are latched into the initial register. These control states propagate sequentially through the chain on each clock edge. Upon reaching the final register stage, the system recognizes that the division algorithm has concluded and the result is ready for Write Back. Effectively, this shift register functions as a synchronized delay line, preserving and tracking the control state of the division instruction throughout its execution.

To maintain data integrity, a `div_stall` signal is asserted by comparing the source registers of the incoming instruction in the Decode stage against the in-flight destination register currently tracked within the shift register chain. Because the division algorithm executes entirely within the isolated `M_ALU`, it does not monopolize the primary ALU. As a result, subsequent, data-independent instructions can continue to issue and execute concurrently. However, the architecture must resolve one structural hazard: when the division operation reaches its penultimate cycle, the primary datapath is forced to freeze for exactly one cycle. This guarantees the Execute (EX) stage is free, allowing the completed division result to transition safely into the Memory (MEM) stage without collision. Furthermore, the tracking module incorporates a dedicated flush mechanism to resolve Write-After-Write (WAW) hazards. If a newer instruction targets the identical destination register as the ongoing division operation, the in-flight division is flushed, as its pending result has been rendered obsolete by the newer instruction.

```
1 typedef struct packed {  
2     logic done ; // also is_div value (done means is div shifted 33  
3     stages so division is done)  
4     logic rd_we ;
```

```

4         logic [4:0] rd ;
5         logic [1:0] rd_in_choose ;
6     } div_control_t ;
7
8     module shift_register(
9         input  logic clk,
10        input  logic rst,
11        input  logic invalid_decode,
12        input  logic [4:0] rs1,
13        input  logic [4:0] rs2,
14        input  logic [4:0] rd,
15        input  div_control_t cur_ctrl,
16        output div_control_t div_ctrl,
17        output logic div_stall
18    );
19        logic ['DIVIDER_STAGES:0] is_stall ;
20        div_control_t stage_ctrl ['DIVIDER_STAGES+1] ;
21
22        always_comb
23        begin
24            for(int i=0 ; i < 'DIVIDER_STAGES-1 ; i++)
25            begin
26                is_stall[i] = ((stage_ctrl[i].done == 1) && (invalid_decode ==
27                0) &&
28                                ((rs1 == stage_ctrl[i].rd) || (rs2 ==
29                stage_ctrl[i].rd))) ;
30            end
31            is_stall['DIVIDER_STAGES-1] = (stage_ctrl['DIVIDER_STAGES-1].done ==
32            1) ;
33        end
34
35        // Case forwarding: div forwarding must forward at stage WB, so need to
36        // stall 1 clk if neccessary
37        assign is_stall['DIVIDER_STAGES] = ((stage_ctrl['DIVIDER_STAGES].done ==
38        1) &&
39                                (invalid_decode == 0) &&
40                                ((rs1 == stage_ctrl['DIVIDER_STAGES].rd) ||
41                                (rs2 == stage_ctrl['DIVIDER_STAGES].rd))) ;
42
43        always_ff @(posedge clk)
44        begin
45            if(rst)
46            begin
47                for(int i=0 ; i < 'DIVIDER_STAGES+1 ; i++)
48                stage_ctrl[i] <= 9'b0 ;
49            end
50            else
51            begin
52                // ctrl of div inst will be shifted to create the delay
53                if(!(is_stall) && (cur_ctrl.done == 1))
54                    stage_ctrl[0] <= cur_ctrl ;
55            end
56        end
57    endmodule

```

```

50         else stage_ctrl[0] <= 9'b0 ;
51         for(int i=1 ; i < 'DIVIDER_STAGES+1 ; i++)
52             begin
53                 // check if rd of following is the same as div above
54                 // -> flush div inst (if div not finish)
55                 if(rd == stage_ctrl[i-1].rd) stage_ctrl[i] <= 9'b0 ;
56                 else stage_ctrl[i] <= stage_ctrl[i-1] ;
57             end
58         end
59     end
60     assign div_ctrl = stage_ctrl['DIVIDER_STAGES] ;
61     assign div_stall = |(is_stall) ;
62 endmodule

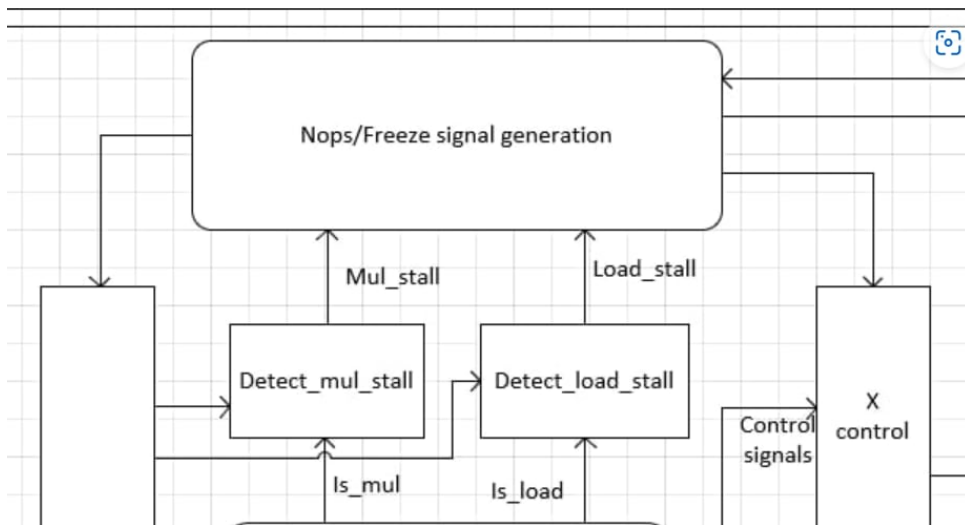
```

5 Data Hazards

Hazard Detection and Forwarding Unit

The implementation of a pipelined architecture inherently introduces data and control hazards, which are significantly complicated by the integration of multi-cycle M-extension operations. To mitigate these bottlenecks and sustain a high Instructions-Per-Cycle (IPC) throughput, the architecture employs a comprehensive hazard detection and resolution unit.

- **Data Forwarding:** To resolve Read-After-Write (RAW) data hazards without introducing stall penalties, the datapath contains a combinational logic block that generates the control signals used to select the correct data for Operand 1 and Operand 2. An operand will select data forwarded from the Memory (MEM) stage if the destination register in the MEM stage matches either source register in the Execute (EX) stage. An essential prerequisite for this forwarding is that the Register File write-enable signal in the MEM stage must be asserted. Data forwarding from the Write Back (WB) stage operates under similar conditions. Notably, if both the MEM and WB stages attempt to forward data to the same source register simultaneously, the data forwarded from the MEM stage is strictly prioritized, as it represents the most recent instruction in the program sequence.
- **Stalling:** While forwarding resolves most data dependencies, standard load-use hazards necessitate a mandatory one-cycle stall (pipeline interlock) to allow the Data Memory to return the requested value. Additionally, in this design, the results from multiplication and division operations are also forwarded from the WB stage. Therefore, dedicated `mul_stall` and `div_stall` signals are implemented to strictly control the program flow and ensure the pipeline stalls correctly when waiting for these multi-cycle forwarding values.



6 Simulation and Implementation

6.1 Simulation

Testbench

I have found and download riscv-test repository in Github. The tests that I'm aiming for test the design in this repository is rv32-ui-p and rv32-um-p which include tests for all basic intructions (rv32-ui) and M-extended instructions (rv32-um), this design is using physical memory so there is "-p" in the postfix of test

```
1 task automatic run_test(input string hex_file, input int num);
2     for(int i=0 ; i<$size(dut.memory.mem_array) ; i++) dut.memory.
   mem_array[i] = 'b0 ;
3     for(int i=0 ; i<$size(dut.DMEM.mem_array) ; i++) dut.DMEM.mem_array[
   i] = 'b0 ;
4     $readmemh(hex_file, dut.memory.mem_array) ;
5     $readmemh(hex_file, dut.DMEM.mem_array) ;
6     //Load data section into DMEM for load test, the reduntant
   instruction load to DMEM don't affect the test
7
8     rst = 1'b1 ; clock(2) ; rst = 1'b0 ;
9     for(int i=0 ; i<num & !dut.datapath.halt ; i++)
10    begin
11        clock(1) ;
12    end
13    clock(3) ;
14
15    expect_rf(32'h1, 3) ;
16 endtask
```

Result

```
=====TESTCASE: SIMPLE TEST=====
```

```
TEST PASS
```

```
=====TESTCASE: ADD=====
```

```
TEST PASS
```

```
TEST PASS
```

```
=====TESTCASE: SUB=====
```

```
TEST PASS
```

```
=====TESTCASE: AND=====
```

```
TEST PASS
```

```
TEST PASS
```

```
=====TESTCASE: OR=====
```

```
TEST PASS
```

```
TEST PASS
```

```
=====TESTCASE: XOR=====
```

```
TEST PASS
```

```
TEST PASS
```

```
=====TESTCASE: SLT=====
```

```
TEST PASS
```

```
TEST PASS
```

```
TEST PASS
```

```
TEST PASS
```

=====TESTCASE: SLL=====

TEST PASS

TEST PASS

=====TESTCASE: SRL=====

TEST PASS

TEST PASS

=====TESTCASE: SRA=====

TEST PASS

TEST PASS

=====TESTCASE: LUI=====

TEST PASS

=====TESTCASE: AUIPC=====

TEST PASS

=====TESTCASE: BEQ=====

TEST PASS

=====TESTCASE: BNE=====

TEST PASS

=====TESTCASE: BLT=====

TEST PASS

TEST PASS

=====TESTCASE: BGE=====

TEST PASS

TEST PASS

=====TESTCASE: JUMP=====

TEST PASS

TEST PASS

=====TESTCASE: LB=====

TEST PASS

TEST PASS

=====TESTCASE: LH=====

TEST PASS

TEST PASS

=====TESTCASE: LW=====

TEST PASS

=====TESTCASE: SB=====

TEST PASS

=====TESTCASE: SH=====

TEST PASS

=====TESTCASE: SW=====

TEST PASS

=====TESTCASE: LOAD-STORE=====

TEST PASS

TEST PASS

=====TESTCASE: DIV=====

TEST PASS

TEST PASS

=====TESTCASE: REM=====

TEST PASS

TEST PASS

=====TESTCASE: MUL=====

TEST PASS

TEST PASS

TEST PASS

TEST PASS

6.2 Implementation

Timing summary

Design Timing Summary			
Setup	Hold	Pulse Width	
Worst Negative Slack (WNS): 0.007 ns	Worst Hold Slack (WHS): 0.037 ns	Worst Pulse Width Slack (WPWS): 2.750 ns	
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns	
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints:	0
Total Number of Endpoints: 7860	Total Number of Endpoints: 7860	Total Number of Endpoints:	3755
All user specified timing constraints are met.			

Utilization

Utilization			
		Post-Synthesis	Post-Implementation
		Graph Table	
Resource	Utilization	Available	Utilization %
LUT	5076	53200	9.54
LUTRAM	100	17400	0.57
FF	3591	106400	3.38
BRAM	8	140	5.71
DSP	4	220	1.82
IO	4	125	3.20
BUFG	1	32	3.13

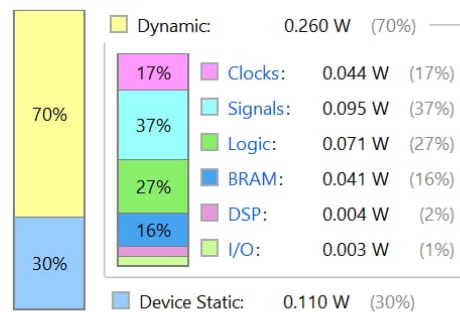
Power on chip

Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: 0.37 W
Design Power Budget: Not Specified
Process: typical
Power Budget Margin: N/A
Junction Temperature: 29.3°C
 Thermal Margin: 55.7°C (4.7 W)
 Ambient Temperature: 25.0 °C
 Effective θ_{JA} : 11.5°C/W
 Power supplied to off-chip devices: 0 W
 Confidence level: Medium
[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

On-Chip Power



7 Conclusion

This project successfully demonstrates the design and implementation of a functionally complete 32-bit, five-stage pipelined RISC-V processor featuring the M-extension. Key architectural highlights include a synchronized shift-register control unit to manage multi-cycle division operations without unnecessarily stalling independent instructions. The robustness and functional accuracy of the processor were rigorously verified, with the design successfully passing all `rv32-ui-p` and `rv32-um-p` test suites from the official `riscv-tests` repository. Furthermore,

the physical implementation met all requisite hardware timing constraints, culminating in a highly optimized, strictly verified, and deployment-ready processor core.

References

- [1] David A. Patterson and John L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface: RISC-V Edition*.
- [2] RVCoreP-32IM: An effective architecture to implement mul/div instructions for five stage RISC-V soft processors [<https://arxiv.org/pdf/2010.16171>].
- [3] riscv-test [<https://github.com/riscv-software-src/riscv-tests>].